

The window, the location, and Selecting Elements

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What is `window`?

- The **global object** for browser JavaScript
- Represents the browser **tab**
- Holds `document`, `location`, `history`, `localStorage`
- Provides `setTimeout`, `alert`, `addEventListener`, and more

```
window === this;      // true (global script)
window === globalThis; // true (browser)
```

Globals, Size, and Scroll

- `var` globals become properties of `window`; `let` / `const` do not
- Prefer `const` / `let` and modules; don't pollute the global scope

```
var greeting = "hi";
window.greeting;    // "hi"

window.innerWidth;  // viewport width in px
window.innerHeight; // viewport height in px
window.scrollTo;    // vertical scroll offset

window.scrollTo({ top: 500, behavior: "smooth" });
```

Other **window** Members

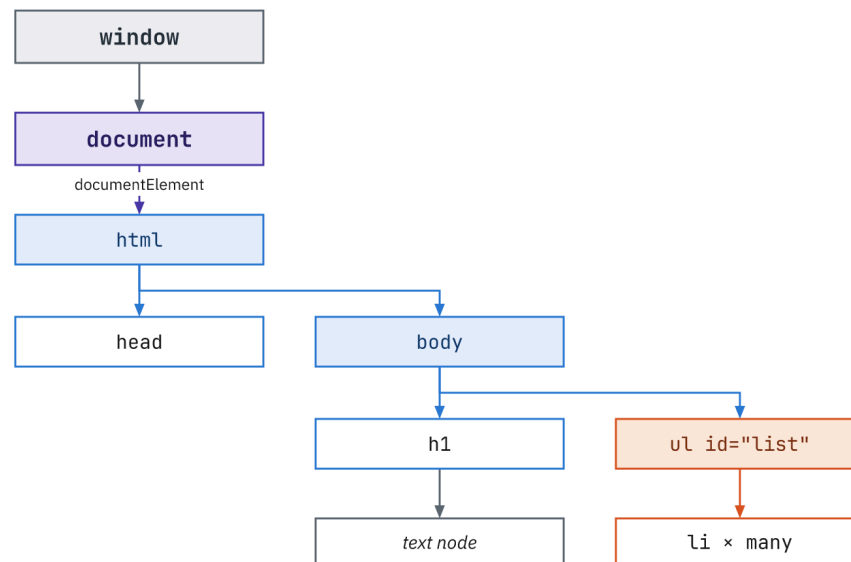
- **navigator**: browser and environment info
- **localStorage** / **sessionStorage**: string key/value storage
- **alert**, **confirm**, **prompt**: simple blocking dialogs

```
localStorage.setItem("theme", "dark");  
localStorage.getItem("theme");    // "dark"  
localStorage.getItem("missing"); // null  
  
console.log(navigator.onLine);    // true / false
```

What is **document** ?

The DOM Tree

The browser parses your HTML into a tree of objects. JavaScript changes the tree, not the file.



Reaching into the tree

```
getElementById('list')
querySelector('#list li')
querySelectorAll('li')
closest('ul')
```

`querySelectorAll` returns a static `NodeList`: a snapshot, not live.

window is the global object

```
window.document
window.location
window.navigator
window.localStorage
```

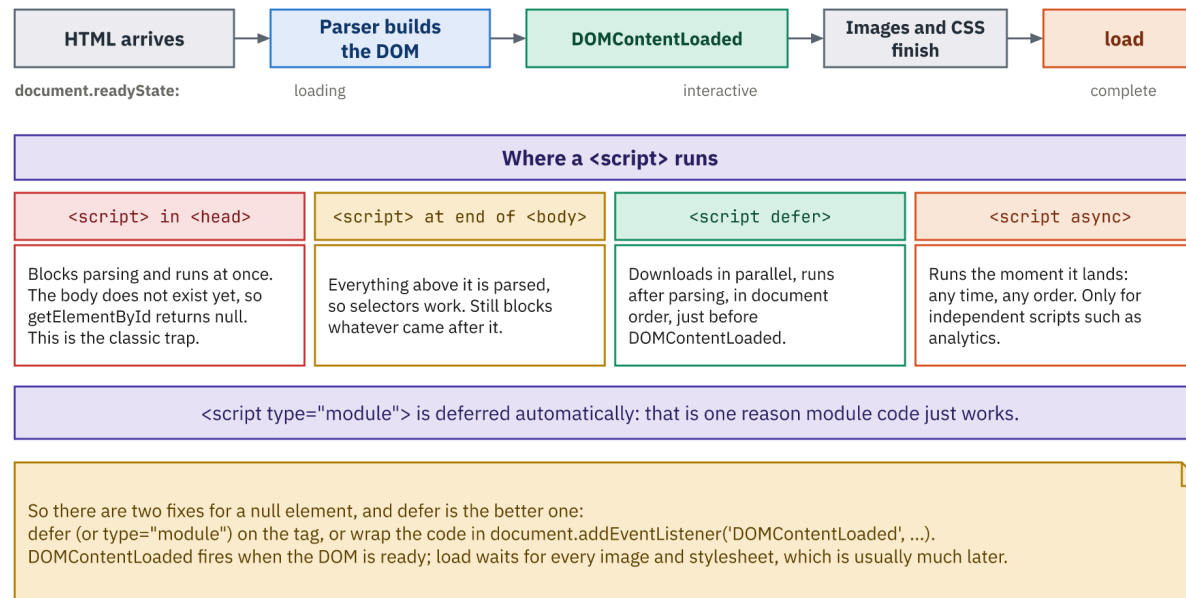
Everything global hangs off it, so `window` is usually optional.

- The **entry point to the page content**: **documentElement**, **head**, **body**
- JavaScript changes the tree in memory, never the HTML file

Running Code When Ready

Page and Script Lifecycle

Why the element was null: your script ran before the browser had parsed it.



- A `<script>` in `<head>` runs before `<body>` exists, hence the `null`
- Fix it with `defer`, `type="module"`, or a `DOMContentLoaded` listener

Reading the URL

- `location` (same as `window.location`) describes the current URL

```
// https://example.com:8080/shop?category=books#reviews
location.href;      // full URL
location.protocol;  // "https:"
location.host;      // "example.com:8080"
location.pathname;  // "/shop"
location.search;    // "?category=books"
location.hash;      // "#reviews"
location.origin;    // "https://example.com:8080"
```

Query Parameters

- `location.search` holds the raw query string; parse it, don't split it

```
const params = new URLSearchParams(location.search);

params.get("category"); // "books"
params.get("missing");  // null
params.has("page");     // false
params.getAll("tag");    // every value for a repeated key

params.forEach((v, k) => console.log(k, "=", v));
```

Navigating

- `href` / `assign`: go to a URL (Back returns)
- `replace`: swap the current entry (Back skips it)
- `reload`: reload the current page, **no arguments**

```
location.href = "https://example.com/profile";  
location.assign("https://example.com/profile");  
location.replace("https://example.com/login"); // no back  
location.reload();
```

querySelector

- Returns the **first** element matching a CSS selector
- Works with `#id`, `.class`, `tag`, or any CSS selector

```
document.querySelector("#main-title");  
document.querySelector(".highlight");  
document.querySelector("button");  
document.querySelector(".menu .item.active");
```

- Returns `null` when nothing matches; guard before using it

querySelectorAll

- Returns **all** matches as a static NodeList
- Has `forEach`; convert with `Array.from` for `map` / `filter`

```
const todos = document.querySelectorAll(".todo");
console.log(todos.length);

todos.forEach((item, i) => console.log(i, item.textContent));

const arr = Array.from(todos).map((el) => el.textContent);
```

Older Selection Methods

- `getElementById`: one element by id (no `#`)
- `getElementsByClassName` / `getElementsByTagName`: **live** collections

```
document.getElementById("user-card");  
document.getElementsByClassName("note"); // live HTMLCollection  
document.getElementsByTagName("div");    // live HTMLCollection
```

- **Live** means the collection updates itself as the DOM changes
- `querySelectorAll` gives a **static** snapshot, usually what you want

Creating and Inserting

- `createElement` builds a node; `textContent` fills it; then append

```
const list = document.getElementById("list");
const frag = document.createDocumentFragment();

for (let i = 1; i <= 5; i++) {
  const li = document.createElement("li");
  li.textContent = "Item " + i;
  frag.appendChild(li);
}
list.appendChild(frag); // one append, not five
```

Geolocation: Ask, Don't Assume

- Lives on `navigator.geolocation`, another `window` API
- Needs **user permission** and a **secure context** (https or `localhost`)
- It cannot return a value, because the browser has to prompt first

```
if ("geolocation" in navigator) {  
  console.log("Geolocation is supported");  
}
```

- So it takes **callbacks**, the first async API in the course

Geolocation: `getCurrentPosition`

```
navigator.geolocation.getCurrentPosition(
  (position) => {
    const { latitude, longitude, accuracy } = position.coords;
    console.log(latitude, longitude, accuracy);
  },
  (error) => {
    // A declined prompt is EXPECTED - warn, don't error
    console.warn("Unavailable:", error.message);
  },
  { timeout: 10000, enableHighAccuracy: false },
);
```

- The error callback is **not** optional; users refuse